# ALU8bit

## Setup Time Violation and Pipelining Optimization Report

**Ahmed Wael Mohammed**

## 1. Part 1: Setup Time Violation Analysis

### 1.1 Timing Report Evaluation

During the synthesis and timing analysis of the 8-bit shift-and-add multiplier unit in Synopsys Design Compiler, 4 setup time violations were observed at clock frequency= 184.162 MHz (i.e CLK_PERIOD=5.43ns). The maximum frequency of the original design is 183.8235 MHz. I used two decimal digits since the tool uses that in its calculations.

The worst-case propagation delay occurred between the clock pin of the launch register `u_mul/a_reg_reg[3]/CK` and the data pin of the capture register `u_mul/product_reg[15]/D`.

**Timing Constraints & Slack Calculation**

$$
\begin{aligned}
t_{\text{arrival}} &= 5.09\,\text{ns} \\
t_{\text{required}} &= 5.02\,\text{ns} \\
\text{Slack} &= t_{\text{required}} - t_{\text{arrival}} \\
\text{Slack} &= 5.02\,\text{ns} - 5.09\,\text{ns} = -0.07\,\text{ns} \quad \text{(VIOLATED)}
\end{aligned}
$$

In a single clock cycle, the signal must propagate through the partial product generation (AND arrays) and cascade through a dense, 7-stage sequential adder chain to resolve the final 16-bit Most Significant Bit (MSB).

Gate-Level Timing Report (Critical Path)

```
Startpoint: u_mul/a_reg_reg[3]
              (rising edge-triggered flip-flop clocked by MASTER_CLK)
Endpoint: u_mul/product_reg[15]
            (rising edge-triggered flip-flop clocked by MASTER_CLK)
Path Group: MASTER_CLK
Path Type: max

Des/Clust/Port     Wire Load Model       Library
------------------------------------------------
alu8_top           tsmc13_wl30           scmetro_tsmc_cl013g_rvt_ss_1p08v_125c

Point                                                                  Incr       Path
-----------------------------------------------------------------------------------
```

```
clock MASTER_CLK (rise edge)                                          0.00       0.00
clock network delay (ideal)                                           0.00       0.00
u_mul/a_reg_reg[3]/CK (DFFRHQX8M)                                     0.00       0.00 r
u_mul/a_reg_reg[3]/Q (DFFRHQX8M)                                      0.48       0.48 r
u_mul/U236/Y (BUFX32M)                                                0.22       0.71 r
u_mul/U234/Y (NAND3X12M)                                              0.15       0.86 f
u_mul/U233/Y (NAND2X12M)                                              0.15       1.01 r
u_mul/U237/Y (NAND3X12M)                                              0.18       1.19 f
u_mul/U240/Y (NOR2X12M)                                               0.21       1.40 r
u_mul/U502/Y (NOR2X12M)                                               0.09       1.49 f
u_mul/U210/Y (CLKXOR2X16M)                                            0.27       1.76 r
u_mul/U209/Y (AO21X8M)                                                0.22       1.98 r
u_mul/U99/Y (INVX16M)                                                 0.09       2.07 f
u_mul/U206/Y (AOI211X4M)                                              0.49       2.56 r
u_mul/U208/Y (OAI32X4M)                                               0.29       2.85 f
u_mul/U6/Y (BUFX18M)                                                  0.22       3.07 f
u_mul/U216/Y (XNOR3X8M)                                               0.49       3.56 r
u_mul/U220/Y (NAND2X12M)                                              0.13       3.69 f
u_mul/U534/Y (CLKXOR2X16M)                                            0.26       3.95 r
u_mul/U539/Y (CLKINVX40M)                                             0.11       4.06 f
u_mul/add_0_root_add_0_root_add_135/B[13] (mult_unit_DW01_add_8)
                                                                      0.00       4.06 f
u_mul/add_0_root_add_0_root_add_135/U35/Y (NAND2X12M)
                                                                      0.11       4.17 r
u_mul/add_0_root_add_0_root_add_135/U34/Y (NAND2X12M)
                                                                      0.09       4.26 f
u_mul/add_0_root_add_0_root_add_135/U28/Y (AOI2B1X8M)
                                                                      0.22       4.47 r
u_mul/add_0_root_add_0_root_add_135/U31/Y (OAI21X8M)
                                                                      0.15       4.62 f
u_mul/add_0_root_add_0_root_add_135/U30/Y (CLKXOR2X16M)
                                                                      0.24       4.86 r
u_mul/add_0_root_add_0_root_add_135/SUM[15] (mult_unit_DW01_add_8)
                                                                      0.00       4.86 r
u_mul/U197/Y (MXI2X12M)                                               0.12       4.98 f
u_mul/U69/Y (INVX20M)                                                 0.11       5.09 r
u_mul/product_reg[15]/D (DFFRHQX1M)                                   0.00       5.09 r
data arrival time                                                                5.09

clock MASTER_CLK (rise edge)                                          5.43       5.43
clock network delay (ideal)                                           0.00       5.43
clock uncertainty                                                    -0.20       5.23
u_mul/product_reg[15]/CK (DFFRHQX1M)                                  0.00       5.23 r
library setup time                                                   -0.21       5.02
data required time                                                               5.02
-------------------------------------------------------------------------------
data required time                                                               5.02
data arrival time                                                               -5.09
-------------------------------------------------------------------------------
slack (VIOLATED)                                                                -0.07
```

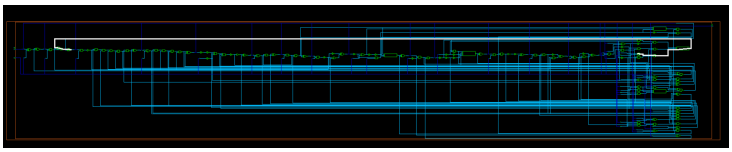


Figure 1: Critical Path in the U_MUL block (highlighted by white marker) from launch to capture register

# 2. Part 2: Datapath Pipelining & Synchronization

## 2.1 Architecture Modification

To resolve the setup violation and maximize the operating frequency ($F_{max}$), a 2-stage pipelined architecture was introduced to replace the purely combinational 1-cycle calcu-

lation. Specifically, I inserted a register between level 3 and level 4 to cut the cascade AND Path, as shown in Figure 1.

**Data Coherency: Why Pipeline the Registered A, B, and Enable?**

**Q: Why register the operands (`a_pipe`, `b_pipe`) alongside the intermediate math?**
If only the 16-bit intermediate sum (level 3) was registered to break the critical path, a data coherency issue would occur. If a new calculation begins on Cycle 2, the unpipelined inputs would overwrite the operands required to finish Cycle 1's computation. By passing the unconsumed inputs forward (`a_pipe` and `b_pipe[7:4]`), the second half of the partial products evaluate using the correct temporal data, ensuring perfect mathematical coherency.

**Q: Why pipeline the enable signal (`en_pipe`)?**
In the original design, the math took 1 cycle. In the pipelined design, the math takes 2 cycles. Pipelining the enable signal ensures the `valid` flag travels down the datapath side-by-side with the data, asserting synchronously only when the 2-cycle calculation is complete.

### 2.2 Post-Synthesis Results

By dividing the combinational logic depth in half, the data arrival time is drastically reduced, safely satisfying the setup time constraints and yielding zero slack (absolute $F_{max}$) Specifically, I achieved a maximum frequency of 223.214 MHz (i.e CLK_PERIOD=4.48ns).

## 3. Part 3: Updated RTL Implementation

Listing 1: Pipelined Multiplier Unit (mult_unit.v)

```
// ------------------------------------------------------------
// THE SINGLE PIPELINE STAGE (To Maximize Fmax)
// ------------------------------------------------------------
reg [15:0] level3_pipe; // Holds intermediate sum
reg [7:0]  a_pipe;      // Passes operand A forward
reg [7:4]  b_pipe;      // Passes unconsumed B bits forward
reg        en_pipe;     // Synchronizes the valid flag

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        level3_pipe <= 16'd0;
        a_pipe      <= 8'd0;
        b_pipe      <= 4'd0;
        en_pipe     <= 1'b0;
    end
    else begin
        level3_pipe <= level3;      // Cut the adder chain here
        a_pipe      <= a_reg;       // Pass A forward for PP4-7
```

```
        b_pipe      <= b_reg[7:4];  // Pass B[7:4] forward for
           PP4-7
        en_pipe     <= en_reg;      // Sync the enable signal
    end
end
```

***Note:*** *In the Deliverables, Reports and Results folder with subscript _ are the initial synthesis for the original design while that not annotated are that of the new design.*